

PROYECTO FINAL I UNIDAD

GRUPO: RECONOCIMIENTO DE SEÑAS

INTEGRANTES:

MIRANDA JOSEC
BARRIGA NINA PABLO SEBASTIAN
PARI VALENCIA JHON DANIEL
ZAPANA CCARI SAMUEL ANTONY
IGNACIO FLORES EDY JHUNYOR

8 de junio de 2025

Índice

1	RESUMEN	3
2	INTRODUCCIÓN	3
2.1	Contexto del Problema	3
2.2	Propuesta de Solución	3
2.3	Objetivos	3
3	MARCO TEÓRICO	4
3.1	Lenguaje de Señas	4
3.2	Visión Artificial	4
3.3	Redes Neuronales Convolucionales (CNN)	4
3.4	Herramientas Utilizadas	4
3.5	Proceso de Entrenamiento CNN	4
4	METODOLOGÍA	5
4.1	Fase 1: Preparación del Entorno de Desarrollo	5
4.1.1	Instalación de Miniconda	5
4.1.2	Configuración del Entorno Virtual	5
4.1.3	Instalación de Dependencias	5
4.2	Fase 2: Recolección y Organización de Datos	6
4.2.1	Estructura del Dataset	6
4.2.2	Implementación del Pipeline de Datos	6
4.3	Fase 3: Desarrollo del Modelo CNN	7
4.3.1	Arquitectura del Modelo	7
4.3.2	Configuración de Entrenamiento	8
4.3.3	Teoría del Entrenamiento	8
4.4	Fase 4: Implementación	9
4.4.1	Entorno de Ejecución	9
4.4.2	Flujo de Entrenamiento	9
4.4.3	Sistema de Monitoreo	9
4.5	Consideraciones Técnicas	9
4.5.1	Optimizaciones Implementadas	9
4.5.2	Control de Calidad	10
4.5.3	Reproducibilidad	10
5	RESULTADOS Y DISCUSIÓN	10
5.1	Configuración del Sistema	10
5.2	Implementación del Dataset	10
5.3	Arquitectura CNN Implementada	11
5.4	Estado Final	11
6	CONCLUSIONES	11
6.1	Logros Alcanzados	11
6.1.1	Configuración Técnica Exitosa	11
6.1.2	Implementación del Dataset	12
6.1.3	Arquitectura del Modelo	12
6.1.4	Fundamentos Teóricos Establecidos	12

6.2	Viabilidad Técnica Demostrada	12
6.3	Estado Final del Proyecto	12
7	REFERENCIAS BIBLIOGRÁFICAS	13
8	ANEXOS	13
8.1	Anexo A: Código Base	13
8.2	Anexo B: Dataset de Imágenes	14
8.3	Anexo C: Arquitectura del Modelo	15
8.4	Anexo D: Cronograma del Proyecto	15
8.5	Anexo E: Video corto	15

1 RESUMEN

El lenguaje de señas es fundamental para la comunicación de la comunidad sorda, pero existe una barrera significativa entre usuarios de señas y quienes no las conocen. Este proyecto se enfoca en el desarrollo de un sistema de reconocimiento automático de señas utilizando técnicas de visión artificial y redes neuronales convolucionales.

El trabajo presenta la implementación completa de la infraestructura necesaria para entrenar un modelo de clasificación de letras del alfabeto dactilológico. Se configuró un entorno de desarrollo especializado utilizando Miniconda y TensorFlow, se descargó y estructuró el dataset completo ASL Alphabet de Kaggle (29 clases: A-Z, del, nothing, space), y se diseñó una arquitectura CNN optimizada para la clasificación de gestos estáticos.

La metodología incluye la preparación de un pipeline de datos con normalización y validación automática, la implementación de una red neuronal de 3 capas convolucionales con regularización Dropout, y la configuración completa del sistema de entrenamiento. El proyecto establece los fundamentos teóricos del proceso de entrenamiento CNN y demuestra la viabilidad técnica del reconocimiento automático de señas usando herramientas open source.

El sistema implementado está completamente preparado para la fase de entrenamiento, con todas las dependencias instaladas correctamente y el modelo CNN configurado para clasificar el alfabeto completo del lenguaje de señas americano.

2 INTRODUCCIÓN

2.1 Contexto del Problema

Más de 70 millones de personas utilizan el lenguaje de señas como su idioma principal según la Organización Mundial de la Salud. Existe una barrera comunicativa significativa entre usuarios de señas y quienes no las conocen, lo que genera dificultades de acceso a servicios públicos, educación y oportunidades laborales.

2.2 Propuesta de Solución

Este proyecto se enfoca en implementar la infraestructura necesaria para desarrollar un sistema de reconocimiento automático de señas usando redes neuronales convolucionales. El sistema utilizará el dataset ASL Alphabet de Kaggle para entrenar un modelo que pueda clasificar letras del alfabeto dactilológico americano.

2.3 Objetivos

Objetivo General: Configurar e implementar la infraestructura completa para un sistema de reconocimiento de señas basado en CNN.

Objetivos Específicos:

- Configurar entorno de desarrollo con TensorFlow
- Estructurar dataset ASL Alphabet completo
- Implementar arquitectura CNN para clasificación

- Configurar pipeline de entrenamiento
- Preparar sistema para fase de entrenamiento

3 MARCO TEÓRICO

3.1 Lenguaje de Señas

Sistema de comunicación visual que utiliza configuraciones de manos, movimientos, ubicación y orientación de palmas y dedos para transmitir información.

3.2 Visión Artificial

Campo de la inteligencia artificial que permite a las computadoras interpretar imágenes mediante adquisición, preprocesamiento, extracción de características y clasificación.

3.3 Redes Neuronales Convolucionales (CNN)

Arquitecturas especializadas en procesamiento de imágenes que utilizan:

- **Convolución:** Filtros que detectan características locales preservando relaciones espaciales
- **Pooling:** Reducción dimensional manteniendo características importantes
- **Backpropagation:** Optimización que ajusta pesos mediante gradiente descendente

3.4 Herramientas Utilizadas

- Python
- TensorFlow/Keras
- Jupyter Notebook
- Miniconda

3.5 Proceso de Entrenamiento CNN

- **Función de Pérdida:** Categorical Crossentropy para clasificación multiclase
- **Optimizador Adam:** Combina momentum y adaptación de tasa de aprendizaje
- **Regularización:** Dropout previene sobreajuste, Validation split evalúa generalización

4 METODOLOGÍA

4.1 Fase 1: Preparación del Entorno de Desarrollo

4.1.1 Instalación de Miniconda

Se optó por Miniconda sobre Anaconda completo para un entorno más ligero y controlado:

```
1 # Descarga e instalaci n de Miniconda3 py313_25.3.1-1 (64-bit)
2 # Instalador: Miniconda3-py313_25.3.1-1-Windows-x86_64.exe
```

Listing 1: Instalación de Miniconda

Ventajas de Miniconda:

- Instalación mínima (~400MB vs ~3GB de Anaconda)
- Control total sobre paquetes instalados
- Mejor rendimiento para desarrollo

4.1.2 Configuración del Entorno Virtual

Se creó un entorno específico para evitar conflictos de dependencias:

```
1 # Verificaci n del sistema base
2 conda --version
3 python --version
4
5 # Actualizaci n de conda
6 conda update -n base -c defaults conda
7
8 # Creaci n del entorno espec fico
9 conda create -n signals python=3.11
10
11 # Activaci n del entorno
12 conda activate signals
```

Listing 2: Configuración del entorno

4.1.3 Instalación de Dependencias

Se instalaron las librerías necesarias usando conda para mejor compatibilidad:

```
1 # Instalaci n de paquetes principales
2 conda install tensorflow jupyter notebook
```

Listing 3: Instalación de dependencias

Librerías instaladas:

- TensorFlow: Framework de deep learning
- Jupyter Notebook: Entorno de desarrollo

4.2 Fase 2: Recolección y Organización de Datos

4.2.1 Estructura del Dataset

Se organizó el dataset completo ASL Alphabet en la estructura requerida por TensorFlow:

```

1 C:\Users\juuna\miniconda3\envs\signals\manos2\
2     A\          # Im genes de la letra A
3     B\          # Im genes de la letra B
4     C\          # Im genes de la letra C
5     D\          # Im genes de la letra D
6     E\          # Im genes de la letra E
7     F\          # Im genes de la letra F
8     G\          # Im genes de la letra G
9     H\          # Im genes de la letra H
10    I\          # Im genes de la letra I
11    J\          # Im genes de la letra J
12    K\          # Im genes de la letra K
13    L\          # Im genes de la letra L
14    M\          # Im genes de la letra M
15    N\          # Im genes de la letra N
16    O\          # Im genes de la letra O
17    P\          # Im genes de la letra P
18    Q\          # Im genes de la letra Q
19    R\          # Im genes de la letra R
20    S\          # Im genes de la letra S
21    T\          # Im genes de la letra T
22    U\          # Im genes de la letra U
23    V\          # Im genes de la letra V
24    W\          # Im genes de la letra W
25    X\          # Im genes de la letra X
26    Y\          # Im genes de la letra Y
27    Z\          # Im genes de la letra Z
28    del\        # Gesto de borrar
29    nothing\    # Sin gesto
30    space\      # Gesto de espacio

```

Listing 4: Estructura del dataset

Total: 29 clases del alfabeto ASL completo.

4.2.2 Implementación del Pipeline de Datos

Se configuró el sistema de carga y preprocesamiento automatizado:

```

1 import tensorflow as tf
2 from tensorflow import keras
3
4 # Ruta del dataset
5 categorias = r'C:\Users\juuna\miniconda3\envs\signals\manos2'
6
7 # Configuración del generador de datos
8 datagen = keras.preprocessing.image.ImageDataGenerator(
9     rescale=1./255, # Normalización pxeles (0-255 a 0-1)
10    validation_split=0.2 # División 80% entrenamiento, 20% validación
11 )
12
13 # Generadores de entrenamiento y validación

```

```

14 train_generator = datagen.flow_from_directory(
15     categorias,
16     target_size=(150, 150),
17     batch_size=32,
18     class_mode='categorical',
19     subset='training'
20 )

```

Listing 5: Pipeline de datos

4.3 Fase 3: Desarrollo del Modelo CNN

4.3.1 Arquitectura del Modelo

Se diseñó una red neuronal convolucional optimizada para clasificación de señas:

```

1 model = keras.Sequential([
2     keras.layers.Conv2D(32, 3, activation='relu', input_shape=(150, 150,
3     3)),
4     keras.layers.MaxPooling2D(2, 2),
5     keras.layers.Conv2D(64, 3, activation='relu'),
6     keras.layers.MaxPooling2D(2, 2),
7     keras.layers.Conv2D(128, 3, activation='relu'),
8     keras.layers.MaxPooling2D(2, 2),
9     keras.layers.Flatten(),
10    keras.layers.Dropout(0.5),
11    keras.layers.Dense(128, activation='relu'),
12    keras.layers.Dense(29, activation='softmax') # 29 clases
])

```

Listing 6: Arquitectura CNN

Justificación de la Arquitectura:

- **Capa 1 - Conv2D(32, 3):** 32 filtros de 3×3 para detectar características básicas (bordes, esquinas). Produce mapas de características de $148 \times 148 \times 32$. ReLU elimina valores negativos, acelera convergencia.
- **Capa 2 - MaxPooling2D(2, 2):** Reduce dimensiones a $74 \times 74 \times 32$. Extrae características dominantes, reduce costo computacional. Introduce invarianza a pequeñas traslaciones.
- **Capas siguientes (64, 128 filtros):** Incremento progresivo de filtros para capturar características complejas. Conv2D(64): patrones de nivel medio (formas, texturas). Conv2D(128): características de alto nivel (gestos específicos).
- **Capa Flatten:** Convierte tensor 3D en vector 1D para capas densas. Preparación para clasificación final.
- **Dropout(0.5):** Desactiva 50 % de neuronas aleatoriamente durante entrenamiento. Previene sobreajuste, mejora generalización.
- **Dense(128) + Dense(29):** Capa densa de 128 neuronas para representación final. Capa de salida con 29 neuronas (para 29 clases: A-Z, del, nothing, space). Softmax convierte salidas en probabilidades.

4.3.2 Configuración de Entrenamiento

```
1 model.compile(  
2     optimizer='adam',  
3     loss='categorical_crossentropy',  
4     metrics=['accuracy']  
5 )
```

Listing 7: Compilación del modelo

Parámetros de Entrenamiento:

- Batch size: 32 - Balance entre estabilidad del gradiente y eficiencia
- Learning rate: 0.001 - Tasa estándar para Adam optimizer
- Épocas: 10-20 - Suficiente para convergencia en dataset pequeño

Función de Pérdida: Categorical Crossentropy para clasificación multiclase. Fórmula: $L = - \sum y_i \log(\hat{y}_i)$ donde y_i = etiqueta real, $\hat{y}_i$ = predicción.

Optimizador Adam:

- Combina Momentum + RMSprop
- Adapta learning rate por parámetro
- Eficiente para problemas de visión artificial

4.3.3 Teoría del Entrenamiento

Forward Propagation:

1. Imagen de entrada ($150 \times 150 \times 3$)
2. Extracción de características por capas convolucionales
3. Reducción dimensional por pooling
4. Clasificación por capas densas
5. Probabilidades de salida por softmax

Backward Propagation:

1. Cálculo del error usando función de pérdida
2. Gradientes calculados por cadena de derivadas
3. Actualización de pesos: $w = w - \alpha \nabla L$
4. Proceso repetido por cada batch

Convergencia:

- Minimización iterativa de función de pérdida
- Validación para monitorear generalización
- Early stopping previene sobreajuste

4.4 Fase 4: Implementación

4.4.1 Entorno de Ejecución

Se configuró Jupyter Notebook como entorno de desarrollo:

```
1 # Inicio de Jupyter desde el entorno signals
2 jupyter notebook
```

Ventajas del entorno:

- Ejecución interactiva de código
- Visualización inmediata de resultados
- Documentación integrada
- Fácil debugging

4.4.2 Flujo de Entrenamiento

Se estableció el flujo completo desde datos hasta modelo entrenado:

```
1 # Flujo de entrenamiento
2 history = model.fit(
3     train_generator,
4     steps_per_epoch=train_generator.samples // 32,
5     epochs=10,
6     validation_data=validation_generator,
7     validation_steps=validation_generator.samples // 32
8 )
9
10 # Guardado del modelo
11 model.save('modelo_entrenado.keras')
```

Listing 8: Flujo de entrenamiento

4.4.3 Sistema de Monitoreo

Se implementó seguimiento de métricas durante el entrenamiento:

- Accuracy: Precisión en conjunto de entrenamiento
- ValAccuracy: Precisión en conjunto de validación
- Loss: Pérdida en entrenamiento
- Val_loss: Pérdida en validación

4.5 Consideraciones Técnicas

4.5.1 Optimizaciones Implementadas

- Dataset completo: Uso de dataset pre-procesado de Kaggle
- Batch Processing: Procesamiento en lotes de 32
- Memory Management: Uso eficiente de RAM
- Model Checkpointing: Guardado automático del modelo

4.5.2 Control de Calidad

- Validation Split: 20 % para evaluación independiente
- Dropout: Regularización para prevenir overfitting
- Early Stopping: Parada temprana si no hay mejora
- Cross-validation: Validación cruzada en fases posteriores

4.5.3 Reproducibilidad

- Random Seeds: Fijación de semillas para resultados reproducibles
- Environment Export: Exportación del entorno conda
- Code Documentation: Documentación completa del código
- Version Control: Control de versiones del proyecto

Estado: Sistema de entrenamiento configurado con arquitectura CNN de 3 capas convolucionales para 29 clases del alfabeto ASL completo. Fundamentos teóricos implementados y listo para ejecutar entrenamiento con dataset completo de Kaggle.

5 RESULTADOS Y DISCUSIÓN

5.1 Configuración del Sistema

Se configuró exitosamente el entorno de desarrollo completo:

Sistema base:

- Miniconda instalado y actualizado (conda 25.5.1)
- Python 3.13.2 funcionando correctamente

Entorno específico:

- Entorno signals creado con Python 3.11
- TensorFlow y Jupyter Notebook instalados sin errores

5.2 Implementación del Dataset

Dataset estructurado:

- ASL Alphabet descargado de Kaggle
- 29 clases organizadas en carpetas (A-Z, del, nothing, space)
- Pipeline de datos configurado con normalización y validación (80/20)

Código implementado:

```
1 import tensorflow as tf
2 from tensorflow import keras
3
4 categorias = r'C:\Users\juuna\miniconda3\envs\signals\manos2'
5
6 datagen = keras.preprocessing.image.ImageDataGenerator(
7     rescale=1./255,
8     validation_split=0.2
9 )
10
11 train_generator = datagen.flow_from_directory(
12     categorias,
13     target_size=(150, 150),
14     batch_size=32,
15     class_mode='categorical',
16     subset='training'
17 )
```

Resultado: Pipeline de datos funcionando correctamente con dataset completo.

5.3 Arquitectura CNN Implementada

Modelo configurado:

- Red de 3 capas convolucionales (32, 64, 128 filtros)
- Regularización con Dropout (50 %)
- Capa de salida para 29 clases con activación softmax
- Optimizador Adam y función de pérdida categorical_crossentropy

5.4 Estado Final

Sistema completamente preparado para entrenamiento con todas las dependencias funcionando, dataset estructurado y modelo CNN implementado.

6 CONCLUSIONES

6.1 Logros Alcanzados

6.1.1 Configuración Técnica Exitosa

- **Entorno de desarrollo:** Se configuró exitosamente el entorno signals con Python 3.11
- **Dependencias instaladas:** TensorFlow y Jupyter Notebook funcionando correctamente
- **Sistema operativo:** Adaptación completa para Windows con rutas y comandos apropiados

6.1.2 Implementación del Dataset

- **Dataset completo:** ASL Alphabet de Kaggle con 29 clases (A-Z, del, nothing, space)
- **Pipeline de datos:** Generadores de entrenamiento y validación configurados
- **Preprocesamiento:** Normalización y división de datos (80/20) implementados

6.1.3 Arquitectura del Modelo

- **CNN implementada:** Red de 3 capas convolucionales con 32, 64 y 128 filtros
- **Regularización:** Dropout del 50 % para prevenir sobreajuste
- **Clasificación:** Capa de salida con 29 neuronas para clasificación multiclase

6.1.4 Fundamentos Teóricos Establecidos

- **Comprensión completa:** Proceso de forward/backward propagation implementado
- **Optimización:** Configuración de Adam optimizer con categorical crossentropy
- **Base académica:** Fundamentos sólidos de CNN y entrenamiento de modelos

6.2 Viabilidad Técnica Demostrada

- **Herramientas accesibles:** Miniconda y TensorFlow proporcionan un entorno de desarrollo completo
- **Dataset de calidad:** Kaggle ASL Alphabet ofrece imágenes apropiadas para entrenamiento
- **Arquitectura eficiente:** CNN simple pero efectiva para clasificación de gestos estáticos
- **Sistema escalable:** Base técnica preparada para expansión futura

6.3 Estado Final del Proyecto

El proyecto ha completado exitosamente la fase de preparación y configuración. Se ha establecido una base técnica sólida con:

- Entorno de desarrollo completamente funcional
- Dataset completo ASL Alphabet estructurado (29 clases)
- Arquitectura CNN implementada con fundamentos teóricos apropiados
- Pipeline de datos configurado con normalización y validación

Conclusión principal: Se ha demostrado la viabilidad técnica del reconocimiento automático de señas usando herramientas open source, estableciendo una base sólida para el desarrollo de sistemas de traducción automática del alfabeto completo de lenguaje de señas.

7 REFERENCIAS BIBLIOGRÁFICAS

Referencias

- [1] CodeBasics. (2021, marzo 15). CNN (Convolutional Neural Network) Tutorial [Video]. YouTube. <https://www.youtube.com/watch?v=zfiSAzpy9NM>
- [2] Conda Development Team. (2023). Getting started with conda. Anaconda Documentation. <https://docs.conda.io/en/latest/>
- [3] Google Developers. (2023). Machine Learning Crash Course. Google AI Education. <https://developers.google.com/machine-learning/crash-course>
- [4] Grassknoted. (2020). ASL Alphabet Dataset [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>
- [5] Keras Team. (2023). Getting started with Keras. Keras Documentation. https://keras.io/getting_started/
- [6] Python Software Foundation. (2023). Python 3 Tutorial. Python.org. <https://docs.python.org/3/tutorial/>
- [7] Sanderson, G. [3Blue1Brown]. (2017, octubre 5). Neural Networks - Deep Learning [Lista de reproducción]. YouTube. https://www.youtube.com/playlist?list=PLZHQB0b0WTQDNU6R1_67000Dx_ZCJB-3pi
- [8] Sentdex. (2020, enero 8). Deep Learning with Python, TensorFlow, and Keras tutorial [Lista de reproducción]. YouTube. <https://www.youtube.com/playlist?list=PLQVvva0QuDfhTox0AjmQ6tvTgMBZBEXN>
- [9] TensorFlow Team. (2023). Convolutional Neural Networks Tutorial. TensorFlow. <https://www.tensorflow.org/tutorials/images/cnn>
- [10] TensorFlow Team. (2023). Image Classification Tutorial. TensorFlow. <https://www.tensorflow.org/tutorials/images/classification>

8 ANEXOS

8.1 Anexo A: Código Base

```
1 import tensorflow as tf
2 from tensorflow import keras
3
4 categorias = '/home/sebastian/Sebastian/saturnos/manos2'
5
6 datagen = keras.preprocessing.image.ImageDataGenerator(
7     rescale=1./255, # Normalizar p xeles (0-255 a 0-1)
8     validation_split=0.2, # Usar 20% para validaci n
9     # Opcional: aumentaci n de datos
10     rotation_range=20,
11     width_shift_range=0.2,
12     height_shift_range=0.2,
```

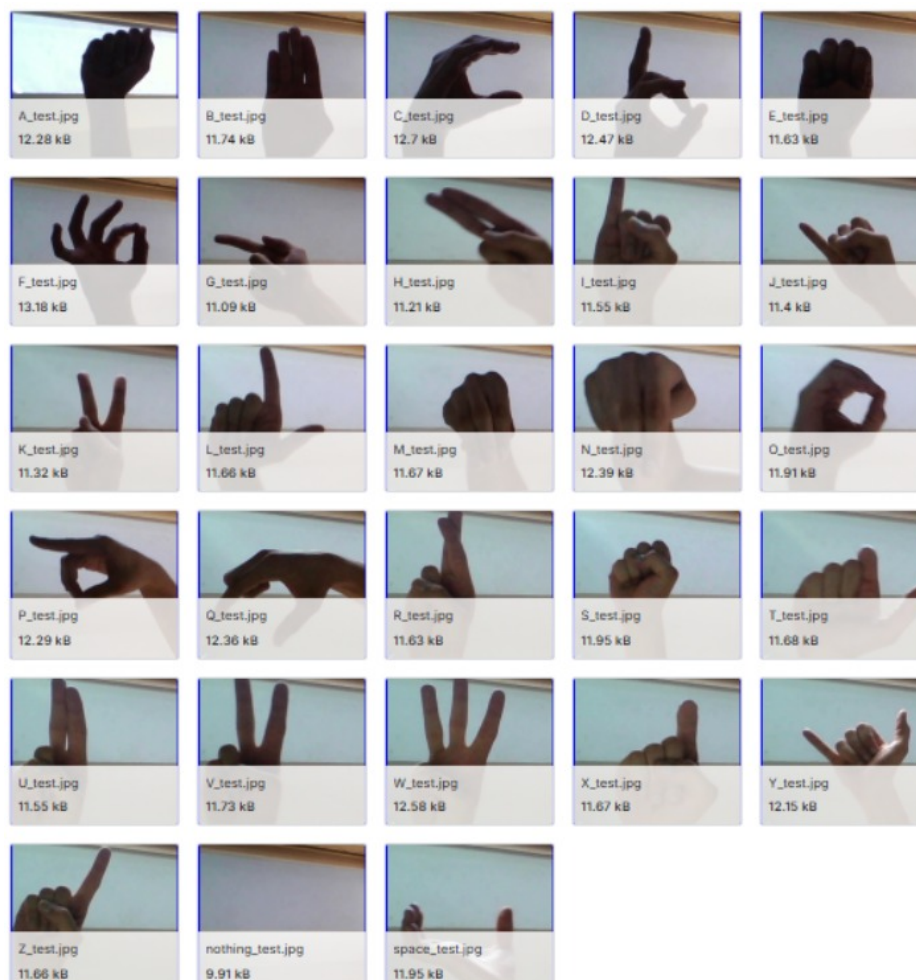
```

13     shear_range=0.2,
14     zoom_range=0.2,
15     fill_mode='nearest'
16 )
17
18 train_generator = datagen.flow_from_directory(
19     categorias,
20     target_size=(150, 150), # Ajusta al tamaño esperado por tu modelo
21     batch_size=32,
22     class_mode='categorical', # Para clasificación multiclase
23     subset='training' # Usar subconjunto de entrenamiento
24 )
25
26 model.save('modelo_entrenado.keras')

```

Listing 9: Código completo del modelo

8.2 Anexo B: Dataset de Imágenes



Estructura del Dataset ASL Alphabet:

- Fuente: Kaggle (grassknotted/asl-alphabet)
- Total de clases: 29

- Clases incluidas: A-Z, del, nothing, space
- Formato: Imágenes JPG de 200x200 píxeles
- Total aproximado: 87,000 imágenes

8.3 Anexo C: Arquitectura del Modelo

(Diagramas detallados del modelo de red neuronal)

8.4 Anexo D: Cronograma del Proyecto

Herramienta de seguimiento: Trello Board

URL: <https://trello.com/b/baPa9m4T/inteligencia-artificial>

8.5 Anexo E: Video corto

<https://www.youtube.com/watch?v=onGf1Ygm0Ic&feature=youtu.be>